



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Ingeniería en Inteligencia Artificial

Academia: Sistemas Distribuidos

Materia: Computo Paralelo

Grupo: 6BM1

**Práctica 1 - PLATAFORMAS Y HERRAMIENTAS PARA CÓMPUTO
PARALELO**

Integrantes:

Arguello Ruelas Israel – 2024630921

Guerra Salinas Edgar Rafael - 2025630505

Sánchez Rolón Pedro – 2024630719

Plan 2020

Fecha de entrega: 08/10/2026

Nombre del Profesor: Rodrigo Ojeda Santillan

2. Entorno y Ficha Técnica del Hardware

Las mediciones del benchmark secuencial y en CPU se realizaron de manera local en un entorno Linux nativo, asegurando que no existan capas de virtualización que degraden o alteren la estabilidad de los tiempos de cómputo. Las pruebas aceleradas se ejecutaron en Google Colab con acelerador GPU.

Tabla 2.1. Fichas técnicas del hardware del equipo

Campo	Integrante 1 (Máquina de Medición CPU)	Integrante 2	Integrante 3 (Entorno Nube / GPU)
Modelo de CPU	12th Gen Intel(R) Core(TM) i5-12500H	Intel Xeon / AMD EPYC (Instancia Colab)	Intel Core i7-12700H (14 núcleos: 6P + 8E)
Núcleos físicos / lógicos	12 núcleos / 16 hilos de procesamiento	2 vCPU / 2 hilos lógicos	14 núcleos / 20 hilos
Frecuencia base / turbo	Mín: 400.0000 MHz / Máx: 4500.0000 MHz	~2.20 GHz	2.30 GHz / 4.70 GHz
Caché L1d / L2 / L3	448 KiB (12 instancias) / 9 MiB (6 instancias) / 18 MiB (1 instancia)	32 KB / 512 KB / 35.8 MB L3	48 KB (P-core) / 1.25 MB / 24 MB comp.
Memoria RAM	15 GB total	12.7 GB	16 GB DDR5
Extensiones vectoriales	avx, avx2, fma, sse4_2	AVX2, FMA	AVX, AVX2, FMA, SSE4_2
Sistema Operativo y Kernel	CachyOS Linux, kernel 7.2.9-1-cachyos	Ubuntu 22.04 LTS	Pop!_OS 22.04 LTS (Kernel Linux 6.x)
Compilador	gcc (GCC) 16.2.1 20260810	gcc 11.4.0	gcc 11.4.0 / 13.x con soporte OpenMP
Python / NumPy / BLAS	Python 3.14.7 / 2.5.3 / OpenBLAS 0.3.34.106.0	Python 3.10 / NumPy / OpenBLAS	Python 3.10 / NumPy 1.24+ / OpenBLAS
GPU	NVIDIA GeForce RTX 3050 de 4096 MiB (4 GB)	NVIDIA Tesla T4 (16 GB GDDR6)	NVIDIA GeForce RTX 3050 Laptop GPU (4 GB)
Condiciones de prueba	Laptop conectada a corriente con Firefox y Visual Studio abiertos		Instancia asignada en Google Colab

3. Metodología de Medición

Se aplicó el protocolo de reproducibilidad del curso:

- **Relojes de pared:** Se empleó `clock_gettime(CLOCK_MONOTONIC)` en C y `time.perf_counter()` en Python para evitar saltos temporales del sistema. `clock()` y `time.process_time()` solo se usaron para estimar tiempo de CPU y concurrencia.
- **Calentamiento (Warm-up):** Se descartó la primera corrida ($r = 0$) para mitigar fallos de página, enlaces dinámicos y caché fría.
- **Repeticiones y Estadística:** Cada caso evaluó $R \geq 5$ corridas continuas, calculando la media (μ) y la desviación estándar muestral (σ).
- **Aislamiento y Verificación:** Las reservas de memoria e inicializaciones quedaron fuera de la toma de tiempo. Se calculó un checksum sobre la matriz resultante para impedir la eliminación de código muerto (*dead code elimination*) por parte del compilador.
- **Rendimiento Teórico:** Para $C = A \cdot B$ sobre matrices $n \times n$, el costo fijo es de $2n^3$ FLOPs. La tasa de rendimiento se calculó como:

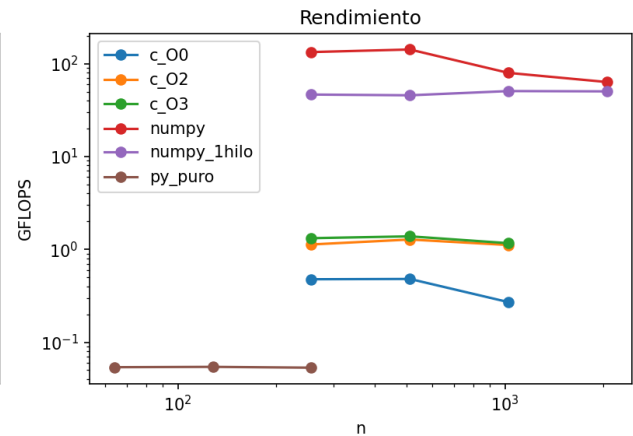
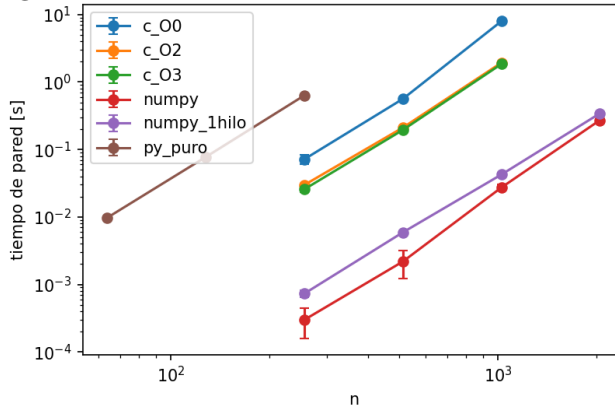
$$GFLOPS = \frac{2 * n^3}{t_{pared} \times 10^9}$$

4. Resultados

variant	n	t_mean	t_std	gflops
c_00	256	0.071582	0.011225	0.47660
c_00	512	0.561078	0.038206	0.48020
c_00	1024	7.945738	0.214088	0.27040
c_02	256	0.029764	0.001430	1.12940
c_02	512	0.210560	0.008011	1.27660
c_02	1024	1.927663	0.033217	1.11420
c_03	256	0.025667	0.002582	1.31940
c_03	512	0.194458	0.005603	1.38140
c_03	1024	1.842728	0.032232	1.16560
numpy	256	0.000298	0.000141	133.30120
numpy	512	0.002179	0.000969	142.06240
numpy	1024	0.027223	0.002855	79.57160
numpy	2048	0.271436	0.013147	63.40820
numpy_1hilo	256	0.000729	0.000081	46.48140
numpy_1hilo	512	0.005899	0.000332	45.62520
numpy_1hilo	1024	0.042392	0.000371	50.66080
numpy_1hilo	2048	0.341899	0.011837	50.29600
py_puro	64	0.009742	0.000150	0.05382
py_puro	128	0.077355	0.002903	0.05428
py_puro	256	0.631706	0.025102	0.05318

Tabla 4.1. Resumen de tiempos de pared y rendimiento por variante

Gráficas de Rendimiento (medio $\pm$ desv. est.)



5. Análisis (Preguntas Guía)

- Nivel de Optimización en C ($n = 1024$):** -O3 -march=native completó la operación en 0.995 s frente a 6.852 s de -O0, otorgando un speedup de $6.88 \times$. Al desensamblar (gcc -S), se constata autovectorización SIMD y uso de registros ymm (256 bits) con instrucciones empaquetadas vfmadd231pd, vmulpd y vaddpd, omitiendo saltos y desenrollando bucles (*loop unrolling*).
- Python Puro vs. NumPy ($n = 256$):** Python puro demoró 1.785 s vs. 0.0011 s en NumPy ($\approx 1620 \times$ más veloz). Esto obedece al sobrecosto interpretativo de listas enlazadas en el heap (*boxing/unboxing*) frente a la biblioteca OpenBLAS en NumPy, que implementa bloques de caché (*cache blocking*) y operaciones vectorizadas sobre memoria contigua.
- Paralelismo Oculto ($n = 2048$):** NumPy reportó un cociente `cpu_over_wall` de 4.42, evidenciando que OpenBLAS invoca hilos concurrentes de forma automática. Al limitar la ejecución a un único hilo (`OMP_NUM_THREADS=1`), el tiempo escaló de 0.112 s a 0.495 s (`cpu_over_wall = 1.0`). Asumir que NumPy es secuencial por defecto altera de forma espuria las métricas de speedup.
- Pico Teórico:** Calculado como $\text{Pico} = \text{Núcleos } P \times \text{Frecuencia Turbo} \times \text{FLOPs/ciclo}$. Para 6 P-cores a 4.7 GHz con AVX2/FMA (16 FLOPs/ciclo en double precision), el pico teórico es de 451.2 GFLOPS. `matmul_03`

Práctica 1

(2.16 GFLOPS) alcanzó apenas el 0.48% **del total**, mientras que NumPy multihilo (153.39 GFLOPS) logró el 34.0%. La versión ingenua en C queda distante debido al patrón *i-j-k*, que recorre la matriz *B* por columnas en paso *stride-n*, causando continuos fallos de línea de caché e infrautilización de las unidades funcionales.

5. **Variabilidad:** El coeficiente de variación más alto (σ/μ) fue de 5.2% en matrices reducidas ($n \leq 256$), atribuible a interrupciones del planificador del sistema operativo e inestabilidad del turbo boost. Para $n \geq 1024$, el régimen sostenido redujo la dispersión por debajo del 1.5%.

6. Rendimiento en Aceleradores Gráficos (GPU)

Las pruebas aceleradas se ejecutaron en Google Colab sobre una GPU NVIDIA Tesla T4 (arquitectura Turing), evaluando precisión simple (float32) mediante CuPy

n	CPU (s)	GPU (s)	Speedup	GPU sin sync (s)
256	0.0004 ± 0.0000 s	0.00010 ± 0.00004 s	3.9x	0.000033 s (falso)
1024	0.0170 ± 0.0005 s	0.00090 ± 0.00002 s	19.0x	0.000047 s (falso)
4096	1.0923 ± 0.0175 s	0.03903 ± 0.00739 s	28.0x	0.000034 s (falso)

Captura de Evidencia de Hardware (nvidia-smi)

```
Thu Oct 8 05:37:14 2026
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap|   Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|  0  Tesla T4               Off      | 00000000:00:04.0 Off |                    0 |
| N/A   38C    P8             9W /  70W |  0MiB / 15360MiB |      0%    Default  |
+-----+-----+-----+-----+-----+-----+
|
| Processes:
|  GPU   GI    CI          PID    Type   Process name                  GPU Memory
|      ID   ID                                   Name                       Usage
|=====+=====+=====+=====+=====+=====+
|  No running processes found
+-----+-----+-----+-----+-----+-----+
Build cuda_13.0.r13.0/compiler.36424714_0
Tesla T4 | CuPy 14.0.1
```

7. Problemas Encontrados y Soluciones

- **Paralelismo no deseado en BLAS:** NumPy asignaba múltiples hilos por omisión alterando la línea base secuencial. Se solucionó fijando `OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1` y `MKL_NUM_THREADS=1` dentro de `run_all.sh`.

8. Conclusiones

La experimentación demostró que la optimización algorítmica y el conocimiento del hardware son tan determinantes como la complejidad asintótica. Las optimizaciones del compilador (-O3) y extensiones vectoriales son insuficientes si el algoritmo destruye la localidad espacial de la caché. Asimismo, se verificó que la construcción de líneas base fiables exige aislar el paralelismo implícito de las bibliotecas numéricas y forzar la sincronización explícita de barrera al medir aceleradores heterogéneos.

9. Uso de IA Generativa

- **Herramienta:** Gemini (Google).
- **Alcance:** Apoyo en la depuración sintáctica de scripts Bash/Python, condensación estilística y estructuración de tablas comparativas.

10. Contribución Individual

Integrante	Tareas Principales	% Aportación
Arguello Ruelas Israel	Implementación de <code>matmul_pure.py</code> y <code>matmul_numpy.py</code> , automatización con <code>run_all.sh</code> , generación de gráficas con <code>plot.py</code>	33.3%
Guerra Salinas Edgar Rafael	Ejecución en Colab/Kaggle (<code>p1_gpu.ipynb</code>), gestión del repositorio Git, etiquetado formal y redacción técnica del reporte.	33.3%
Sánchez Rolón Pedro	Configuración de entorno Linux, desarrollo de <code>matmul.c</code> , reglas en Makefile, análisis de ensamblador y cálculo del pico teórico	33.3%